this is where things traditionally get tricky. How do you generate a valid JWT? Luckily, in .NET 7, the .NET CLI comes with a tool to make creating test tokens easy.

25.4 Using the user-jwts tool for local JWT testing

In section 25.3 you added JWT authentication to your application and protected your API with a basic authorization policy. The problem is that you can't test your API unless you can generate JWT tokens. In production you'll likely have an identity provider such as Auth0, Azure Active Directory, or IdentityServer to generate tokens for you using OpenID Connect. But that can make for cumbersome local testing. In this section you'll learn how to use the .NET CLI to generate JWTs for local testing.

In .NET 7, the .NET CLI includes a tool called user-jwts that you can use to generate tokens. This tool acts as a mini identity provider, meaning that you can generate tokens with any claims you may need, and your API can verify them using signing key material generated by the tool.

TIP The user-jwts tool is built into the software development kit (SDK), so there's nothing extra to install. You need to enable User Secrets for your project, but user-jwts will do this for you if you haven't already. The user-jwts tool uses User Secrets to store the signing key material used to generate the JWTs, which your app uses to validate the JWT signatures.

Let's look at how to create a JWT with the user-jwts tool and use that to send a request to our application.

25.4.1 Creating JWTs with the user-jwts tool